Distributed In-Memory Cache with Strong Consistency Guarantees

(Distributed In-Memory Cache
with Strong Consistency Guarantees)

Dominik Szczepaniak

Praca inżynierska

Promotor: dr Piotr Witkowski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

20 grudnia 2025

Abstract

When designing distributed systems we need to make trade offs. The basics of such trade offs are defined in CAP theorem, where we cannot guarantee three of Consistency, Network partition tolerance and Availability. Most of implementations try to ensure Availability and Network partition tolerance, for maximizing throughput and speed of cache. In this paper, I will focus on Network Partition tolerance and Consistency. I want to achieve Strong Consistency.
My architecture splits the system into layers. First layer is a Control Plane, responsible for managing the cluster and for the consensus. Raft algorithm is used for consensus between the servers in the cluster. The next layer is the Data Plane. It is responsible for splitting data into shards. Each of these shards is called a data node. Another part of the system is Smart Client. It is locally storing the cluster topology, fetched from Control Plane to manually calculate to which data node should one send a request. Thanks to that, our system minimizes network overhead and latency. The system is implemented in Go language with gRPC for internode communication and http for client-server comunnication.
Implementation has been tested with multiple tests. The Raft algorithm was a very detailed and throughoutly tested, as this is the base of the entire system. Then, there were written End-To-End tests, alongside Failure injection, such as network partitions, single servers and nodes failures, stops and restarts. There were also benchmarks, comparing the implementation to known cache systems, such as Redis. Results show that systems meets the conditions of Strong consistency and Network Partition and also offers competitive throughput compared to industry-standard solutions.

Gdy tworzy się systemy rozproszone trzeba wziąć pod uwagę różne kompromisy które trzeba poczynić przy tworzeniu takiego systemu. Podstawowymi kompromisami w takich systemach jest wybór między Spójnością, Dostępnością oraz Tolerancją na podział sieci.
CAP theorem mówi nam, że system nie jest w stanie spełnić jednocześnie wszystkich trzech cech, więc musimy zrezygnować z jednej z nich. Ja w swojej pracy postawiłem na Spójność oraz Tolerancję na podział sieci. Większość rozwiązań produkcyjnych stawia na dostępność zamiast spójności, aby przyspieszyć działanie i ogólny czas dostępu do systemu.
Moja architektura dzieli system na warstwy. Pierwszą z nich jest Control Plane, który jest odpowiedzialny za zarządzanie klasterem oraz konsensus, dzięki algorytmowi Raft, pomiędzy serwerami wchodzącymi w jego skład. Kolejną wartstwą jest Data Plane który ma rozdrobione węzły danych. Waznym elementem systemu jest Smart Client, który zapisuje lokalnie topologię klastra, którą pobiera z Control Plane i dzięki temu wysyła zapytania do odpowiednich węzłów danych, minimalizując opóźnienie i ilość zapytań sieciowych.
Cały system jest zaimplementowany w Go oraz używa gRPC do komunikacji między serwerami oraz http do komunikacji między serwerami a klientami.
Implementacja została przetestowana przeroznymi testami. Bardzo dokładnie został przetestowany algorytm Raft, który jest podstawą całego systemu. Następnie zostały przeprowadzone testy End-to-End, testy na wprowadzanie celowego błędu do systemu, np. podział sieci, restart serwerów. Wykonane zostały również benchmarki, które porównały implementację systemu do popularnych implementacji stosowanych w branży. Wyniki demonstrują, że system spełnia założone wymagania przy wydajnościach porównywalnych do popularnych implementacji.

Contents

Test

Chapter 1

Introduction

1.1 Distributed systems and the problem with data consistency

When we are collecting data from our system the amount might grow infinitely, yet computers have limited amount of RAM and Disk space they can support at once, whether from motherboard limits or just physical limits and law of nature. One day developers might have to come to a decision to start scaling the application. In the beggining, the application might be scaled to just two or three servers, but as more and more data is collected, the entire ecosystem might span tens and hundreds of servers. Some of the data that is collected might be needed often, very often. If we need data often, then we usually put it in cache, as it's fast and we don't want to fetch everything from slow databases. But as the amount of data grows even larger, even the cache cannot keep all the data we need. Then we need to scale cache. And there comes a problem. Assume we have two servers that split the data for cache. One of them have a value 3 for key 1, the other have value 4 for key 1. Which answer is correct? This is the problem of data consistency in the system.

1.2 Objective and scope of the thesis

The goal of thesis is to build a distributed in-memory cache that assures strong consistency of the data - there exist a linearizable moment in the system, where every single server agrees to data they share. As this is cache, and not database, we do not guarantee data durability. If we data is lost, then the user has to refetch data from database and put it in the cache again. We only guarantee, that whenever user asks for the data and receives a response, this response will be correct.

1.3 Technology stack choice

I choose to go with Go for a programming language. This is a language that is based around concurrency which is the base of this thesis. Both single servers are operating concurrently, and they also talk to each other in concurrent way. Go is also very good for network operations, which there are a lot. Other considered choices were C++ and Java. The first was rejected because of it's complexity and the scale of this project aswell as author's skills in using it, the latter because it's slower than Go and less developer friendly.
For the inter-node communication the gRPC was used, because of it's speed. For server-client communication, http was used, because of it's ease of usage.

1.4 Structure of this thesis

Chapter 2 presents theoretical background necessary to understand the problem of state management in distributed systems. It discusses the CAP theorem, data consistency models, consensus algorithms - what are they, why we need them. It will also briefly compare two of the most popular consensus algorithms and describe what the chosen one is doing. It also reviews existing solutions for architectures, sharding, client types and actual systems used in the industry.

Chapter 3 presents more detailed requirements of the project and the system design. It defines high-level architecture, separation of Control Plane from Data plane. It also details communication between servers and the specific design of client.

Chapter 4 presents a detailed implementation of Raft consensus algorithm. It details implementation of basic parts of the system aswell as more advanced.

Chapter 5 covers the implementation details of the remaining system components. It describes the logic of the Control Plane, the Data Nodes and the client library. Special attention is given to mechanisms of shard rebalancing and failure detection.

Chapter 6 is dedidacted to testing and verification of the system. It describes the testing methology, including unit tests, end to end tests, integration tests and failure injection tests used to validate system correctness and stability.

Chapter 7 presents the performance analysis and benchmarks in comparison to other industry-standard systems. It contains the results of benchmark tests measuring latency and throughput (requests per second).

Chapter 8 summarizes the thesis, evaluating the degree to which the project goals were met, discussing the encountered challenges and potential directions for future development.

Chapter 2

Theoretical Foundations and Overview of Solutions

2.1 Consistency models in distributed systems

Distributed systems need to agree on the data they have. Because of how the systems are built, we define three possible consistency models, ranked by their strength:

- Strong consistency - in this model all nodes agree on the order in which operations were done. Reads will always return the most recent version of the data and when update occurs, then this model will make sure that every other server will reflect on that. Because of these guarantees, this model has limitation in terms of how fast it allows the system to work.
- Sequentual consistency - all nodes see all operations in the same global order, but that order is not required to be the actual real-time order. Operations happen in sequential, single, unifed order.
- Casual consistency - ensures all nodes see the correct order for operations that are casually related, i.e. one operation is influenced by another
- Eventual consistency - The most relaxed model, it prioritizes availability and performance. It allows that data between server might be conflicting, but if no updates occur, eventually these conflicts will resolve.

2.1.1 The CAP theorem and trade-offs

CAP theorem, formulated by Eric Brewer in [1], says that a networked system that shares-data, can strictly provide only two of three properties:

- **Consistency (C):** Reads will receive the most recent write or an error. If a

system is consistent, then all nodes will see the same data at the same time, just like a single-node database would.

- **Availability (A):** Request will always receive a non-error response, without the guarantee that it can contain untrue data. The base of availability is that the system remains operational for reads and updates, even if some nodes fail.
- **Partition Tolerance (P):** The system will work even if a network failure occurs or some nodes crash or there are very big delays.

In real-world distributed systems, we cannot have Consistent and Available system, because we cannot guarantee that network will not partition. If a hardware failure or network failure occurs, then there is inevitable Network Partition. Hence in real-world systems, we will choose between Consistency and Availability. Most cache systems focus on Availability, because we want to maximize speed, and if we want consistency, we should use database. However, there are cases, where we need Consistency for our cache, for example in banking applications.

So the systems are either:

1. **CP (Consistency + Partition Tolerance):** When partition occurs, system will shut down all non-consistent nodes or refuse writes if the system cannot support them any longer. This approach proritizes safety over throughput or uptime.
2. **AP (Availability + Partition Tolerance):** When a partition occurs, system will continue to accept write and read requests, with probability that it will return untrue data. This approach proritizes throughput, uptime and user-experience over corectness of the data.

So my system will be a CP system.

2.1.2 Strong Consistency (Linearizability) vs. Eventual Consistency

Strong Consistency (Linearizability)

Linearizability is the strongest guarantee for consistency for single-object requests. It gives a ilussion that system is in fact a single copy of data and the data is not distributed. It gives us a guarantee that once write operation is completed sucessfully, all subsequent read operations will return that value, or a newer value if any update happened.

Linearizability (the formal definition of Strong Consistency) says:

- If operation A completes (client gets OK), then every operation B that starts after A finishes must see the effect of A.

- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," as long as the system is consistent about that choice from then on.

Mathematically, when operation $W(x)$ completes at time t_1, then for any given operation $R(x)$ starting at time $t_2 > t_1$ must see the effects of $W(x)$.

Eventual Consistency

In eventually consistent systems, if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value. This model allows for temporary inconsistencies where different nodes serve different versions of data. This approach enables high availability and low latency because writes can be accepted by a single node without waiting for other nodes to accept and make a consensus. This approach, however, requires to add logic for conflict resolution strategies when merging data from different nodes.

2.2 Split brain problem

Split-brain is a problem in distributed system where a network partition itself into two or more disconnected graph of nodes. This makes the groups unable to communicate with each other and each group may incorrenctly conclude that the other nodes from different groups have failed. This in consequence may start tasks for each groups to find it's leader or primary node. There are multiple ideas to manage this problem. Raft algorithm for example only allows the node to become a leader if more than half of total nodes agrees. The other possibility is to use technique called fencing. This technique is a mechanism where a node physically disables its peer to ensure that it cannot write to shared storage.

2.3 Consensus algorithms

Consensus algorithm is as the name suggest, algorithm that takes care of the state of the data and make sure than all nodes agree on consensus. It is backbone of Consistent systems.

2.3.1 Paxos vs. Raft – Comparison of understanding and implementation

Paxos is an algorithm created by Leslie Lamport and historically the most used algorithm for distributed consensus because of long it has been available. It was mathematically proven to be safe and efficient. However it's also a very complex to understand algorithm and hard to implement.

Raft is more decent algorithm, from 2014, that was created to be easier to implement than Paxos. It offers the same performance as Paxos, but less safety as not all parts of this algorithm have been formally proven [2], however it's still widely used in many modern distributed systems such as CockroachDB[3], MongoDB[4], Neo4J[5], RabbitMQ[6]. It decomposes the problem of consensus into smaller, independent sub-problems of Leader election and Log replication.

Key Differences:

- **Strong Leader:** Raft imposes a stronger leadership model. Log is always send from leaders down to followers. This simplifies the management of the replicated log compared to Paxos, where data can be send in complex peer-to-peer patterns.

- **Log Continuity:** Raft guarantees that if a follower has an entry at index N, it also has all entries from 1 to $N-1$ identical to the leader. Paxos allows for gaps in the log that must be filled later, increasing implementation complexity.

2.4 Detailed analysis of the Raft algorithm

Raft ensures that a cluster of N servers can tolerate F failures, where $N = 2F + 1$.

Leader Election

Raft uses a heartbeat mechanism to maintain leadership. If no heartbeat is received in some configurable time window, then server can assume the leader is dead and start a new election. The election start by the node incrementing its current `Term` (a logical clock used to detect stale information), changing its current state to the Candidate state, voting for itself, and broadcasting `RequestVote` RPCs to all other nodes. A candidate wins if it receives votes from a majority of the cluster. We want to have randomized timeout for election start because we want to minize the probability of multiple nodes starting election at the same and because of that not one receiving a majority, which would cause system stalls.

Log Replication

Once elected, the leader accepts client commands. A command flows through the following lifecycle:

1. The leader appends the command to its local logs.
2. It sends `AppendEntries` RPCs to all followers.
3. Once a majority of followers acknowledge the entry, the leader marks the entry as **committed**.
4. The leader applies the entry to its state machine and returns the result to the client.
5. In subsequent heartbeats, the leader notifies followers of the new `commitIndex`, instructing them to apply the data to their own state machines.

Safety

Raft ensures safety via the **Election Restriction**. Candidate for winner won't win election if it's log is not in the most up to date state. A candidate will also not win an election if it's **Term** is too old, this is, if it's bigger than 1, because nodes increase term by 1 every time a new leader is elected, so if Term difference is bigger than 2, then the node with lower term must have lost access to the cluster, because he skipped at least one leader.
During the voting process, a node will not vote for a candidate if the candidate's log is not at least as up-to-date as its own (comparing the term and index of the last log entry). This guarantees that committed data is never overwritten, preserving the State Machine Safety property.

2.5 Cache system architectures

Consensus can only take us as long as the coordination of data allows. For the system scalability to handle gigabytes of data and thousands of requests per second we need to use architectural designs.

2.5.1 Sharding and Consistent Hashing

Because we need a cache so big to use multiple servers, we need a way to split data between them. The process of splitting the data is called sharding [7].

Traditional Modulo Hashing

The most basic approach would just use $server = \text{hash}(key) \pmod N$, where N is the number of servers. This might be fine, if we had stable amount of servers and immediately could calculate how much data at most we will use. However if we don't know how much data we need, or if we don't want to pay for servers to use 5% of it's resources we would want to scale on the go.
With the basic approach of modulo, if a server is added or removed (N changes), the result of the modulo operation changes for nearly all keys. This will force us to move data between all of the server, forcing a massive rebalancing of data across the entire cluster. This is just inefficient and way too slow.

Consistent Hashing

Consistent hashing maps both data and servers onto a logical ring topology (0 to $2^{32} - 1$). A key is assigned to the first server encountered moving clockwise on the ring.

When a server is added or removed, only K/N keys need to be remapped (where K is the total keys), which significantly minimizizes data movement.
But this has another problem. What if some part of the ring is used way more often then other? For this case we use **Virtual Nodes**. Each physical server is placed onto the ring multiple times (e.g., at 100 different positions). This statistical distribution ensures a more uniform load balancing across the cluster.

2.5.2 Role of the client: Thin Client vs. Smart Client

Of course the system is build for someone, a client. So there also needs to be made a decision on how the client will connect to the system. There are two broad options:

- **Thin Client (Proxy-based):** The client is unaware of the cluster topology. It sends requests to a load balancer or a random node (acting as a proxy), which then forwards the request to the correct shard.
  - *Pros:* Simple client implementation; topology changes are transparent to the application.
  - *Cons:* Additional network hop (Client → Proxy → Shard), increasing latency.
- **Smart Client:** The client maintains a local map of the cluster topology (slots mapping to node IP addresses). It computes the hash and connects directly to the target node.
  - *Pros:* Lowest possible latency (single hop); no bottleneck at a central proxy.

  - *Cons:* Complex client logic; the client must listen for cluster configuration changes (e.g., failovers or resharding) to update its internal routing table.

2.6 Overview of existing solutions (Redis Cluster, Etcd) – Analysis in terms of consistency

To contextualize the solution developed in this thesis, it is valuable to analyze how established systems handle these trade-offs.

2.6.1 Redis Cluster

Redis Cluster is the industry standard for distributed caching. It employs a multi-master architecture with asynchronous replication.

- **Architecture:** It uses sharding with 16,384 hash slots. Each master node owns a subset of slots and has one or more replicas.
- **Consistency:** Redis prioritizes Availability and Performance (AP-leaning). By default, replication to slaves is asynchronous. If a master crashes after accepting a write but before replicating it to a slave, and that slave is promoted to master, the write is lost. Redis does not guarantee strong consistency; it provides eventual consistency and high throughput.

2.6.2 Etcd

Etcd is a distributed key-value store used primarily for shared configuration and service discovery (e.g., in Kubernetes).

- **Architecture:** It is built strictly on the Raft consensus algorithm. Unlike Redis, it does not shard data by default; every node replicates the full state machine.
- **Consistency:** Etcd is a CP system guaranteeing strict serializability. Every write goes through the Raft leader and must be committed to a quorum before returning success. This ensures data safety at the cost of lower write throughput compared to Redis.
- **Durability:** Etcd is a system that guarantees durability: Even if a node crash, the data is not lost.

The solution implemented in this thesis (Chapter 4) aligns more closely with the Etcd model regarding consistency (using Raft for strong guarantees) but incorporates

architectural elements of Redis (key-value storage focus) to explore the performance implications of strong consistency in a caching scenario. My solution **does not** guarantee durability, hence if the node crashes, the data needs to be refetch from durable data storage. This is one of the biggest differences between Etcd.

Chapter 3

Requirements analysis and Software Architecutre Design

3.1 Functional and non-functional requirements

We define functional requirements as what the system must do, specifying it's behaviours and functions:

- Support GET, PUT, DELETE operations via HTTP api
- Automatic sharding of key-space across a configurable number of shards
- Automatic replication of each shard (primary + replicas)
- Dynamic cluster membership - data nodes can be added and removed at runtime
- Rebalancing when nodes join, leave or fail
- Smart client that routes requests to correct node and caches topology

We define non-functional requirements as how the system operates, what it guarantees in perms of performance, security, usability and readability:

- Strong Consistency - the system must reject requests when quorum cannot be reached, the system guarantees that all responses are correct
- Partition tolerance - system continues to operate even if some of the nodes fail or disconnect from others
- Fault tolerance - system detects node failures and triggers automatic fail-over
- No downtime when adding or removing new nodes and distributing shards between data nodes

3.2 High level architecture

The system is split into two logical parts. One of the part is Control Plane, the second one is Data Plane.

3.2.1 Separation of Control Plane and Data Plane

Control plane is responsible for:

- Runs a Raft consensus algorithm that stores cluster metadata - topology, shard mapping, node status, epoch counter.
- Exposes internal HTTP endpoints used by DataNodes for pull based shard migration
- Performs node failure detection (via the ‘Reaper‘) and initiates rebalancing

Data Plane is responsible for:

- Stores the actual key-value data
- Handles client RPC (GET, PUT, DELETE)
- Reports heartbeats to the Control Plane

3.2.2 gRPC communication protocol and interface definition

gRPC communication protocol is a RPC framework created by Google. It is a binary-encoded version of basic RPC, allowing it to be faster. To define gRPC functions and data types, protocol buffer files are used. We define 6 methods and 10 data types. All of these functions are based on Raft Algorithm as only these functions are used in gRPC. There are more details about this functions in Chapter 4.

3.3 Design of Control Plane module

3.3.1 Cluster topology and metadata menagement

Topology data contains:

- Nodes map: node ID mapped to NodeMetadata - address, status, last heartbeat timestamps

- Shards map: shard ID mapped to ShardMetadata - primary ID, replica ID, status
- Epoch - monotonically increasing identificator used for conflict-free updates

Controller updates cluster topology using Raft Algorithm. I want to make sure that Control Plane gives me a Strong Consistency, because if it wouldn't, then the system also wouldn't be Strongly Consistent. This is because, if there are conflicts in Control Plane, then the client might send request to wrong datanode, hence creating impossible state, which would not guarantee "correct response on every request".

Rebalancing is using Control Plane's active nodes to determine which node should be primary and which should be a replica. Then it writes new config to Raft Log, that is distributed between every other Controller in Control Plane.

3.3.2 Node failure detection mechanism

I implemented a Reaper mechanism which tracks last heartbeat timestamp of each nodes. In background, Reaper runs a thread that checks if node did heartbeat within configured grace period. If it didn't, then Reaper invokes a callback to Control Plane, triggering a topology rebalance.

3.4 Smart Client design

3.4.1 Query routing strategy and topology caching

The client on initialization connects to Control Plane to fetch a topology that it stores in local cache. On each request, the client hashes the key to shard ID and looks up the primary node from cached topology. Then the request is sent.

3.4.2 Handling retries and Failover mechanism

On RPC failure the client retries the request against current primary. If this primary is unreachable or the request is rejected, then the client will refetch the cluster topology.

Chapter 4

Implementation of the Raft Consensus Module (pkg/raft)

The core of my distributed system is the Raft consensus algorithm. It is responsible for coordinating controllers, that basically do majority of thing in the application. The only place where Raft is not used is the Data Plane, where nodes just accepts GET/PUT/DELETE and Control Plane makes sure that they do it correctly. My own implementation of this algorithm is located in pkg/raft. The algorithm is decomposed into smaller problem of Leader Election and Log Replication. To limit infinite growth the snapshot mechanism is also implemented. There are multiple supporting functions that I will write in next sections.

4.1 Raft node state machine

The basic of Raft algorithm relies on the elected leader. We define leader as a node that won the election, that is, quorum of nodes (more than half) agreed that the node can become a leader. Each node keeps it current state in currentRole field. Node can be in one of three states: Follower, Candidate, Leader.

4.1.1 Implementation of roles: Follower, Candidate, Leader

The roles are defined as typed constants in `types.go`: `Follower`, `Candidate`, and `Leader`.

- **Follower:** The default state upon initialization (in `NewRaft`). Followers passively respond to RPCs from leaders and candidates.
- **Candidate:** A node transitions to this state when an election timeout occurs. This logic is encapsulated within the `startElection` method in `election.go`.

- **Leader:** The node that handles all client requests and coordinates log replication. Transition to this role is handled by the `becomeLeader` function in `utils.go`, which initializes the `Replicator` instances for all peers.

4.1.2 Leader election mechanism (election.go) and term handling (Terms)

The election logic is decoupled into the `Elector` struct located in `election.go`. This component manages the election timer, which is randomized to prevent split votes. The timer duration is calculated in `nextTimeout` based on a minimum and maximum election timeout range (e.g., 400ms to 800ms).

When the timer fires without receiving a valid heartbeat from a leader, the node increments its `currentTerm` and transitions to the Candidate state. The election process involves:

1. Resetting the election timer.
2. Voting for itself (`votedFor` set to `selfID`).
3. Broadcasting `VoteRequest` RPCs to all peers in parallel.

The incoming `VoteRequest` is handled in `raft.go`. The implementation strictly adheres to the Raft term logic: if a request contains a term higher than the node's `currentTerm`, the node immediately steps down to Follower status.

4.2 Log replication logic (replicator.go, log.go)

Once a leader is elected, it must replicate client commands to the cluster. This process is managed by the `Replicator` struct defined in `replicator.go`. The leader maintains a separate `Replicator` instance for every follower, allowing for asynchronous and isolated replication streams.

4.2.1 Log entry structure (LogEntry) and handling of LogRequest (AppendEntries)

The log is represented as a slice of `LogEntry` structs. Each entry contains:

- `Term`: The term when the entry was received.
- `Message`: The actual state machine command (e.g., `PUT`, `GET`, `DELETE`), defined in `messages.go`.

Replication is triggered via the `LogRequest` RPC (standard Raft *AppendEntries*). The `Replicator` runs a continuous loop that monitors a signal channel. When triggered, it calculates the `prevLogIndex` and `prevLogTerm` to ensure log consistency. These arguments are packaged into a `LogRequestArgs` Protocol Buffer message and sent via the `PeerClient` interface.

4.2.2 Committing entries (Commit Index) and applying to the state machine

The leader tracks the `matchIndex` (represented as `ackedLengths`) for each follower. When a majority of nodes acknowledge a log entry, the leader advances its `commitedLength`. This logic is implemented in the `commitLogEntries` function in `raft.go`.

Upon commitment, messages are applied to the local state machine via the `Application` interface (defined in `application.go`). The application executes the command (e.g., updating the in-memory cache) and returns the result, which is then sent back to the client via a response channel.

4.3 Data persistence management

To survive crashes and restarts, the Raft module implements strict persistence guarantees using the `DataSaver` struct located in `persistance.go`.

4.3.1 Writing Raft state to disk (persistance.go)

The persistence layer uses Go's `encoding/gob` package to serialize data structures to disk. The `DataSaver` utilizes a worker pool pattern to handle save requests asynchronously, preventing disk I/O from blocking the main Raft consensus loop.

The state is split into two files:

- **Metadata File:** Stores the `currentTerm`, `votedFor`, and `commitedLength`.
- **Logs File:** Stores the append-only sequence of `LogEntry` objects.

The `SaveValues` method is called whenever the node's term, vote, or log changes, ensuring that the critical Raft state is durable before responding to RPCs.

4.3.2 Recovering state after node restart

During initialization in `NewRaft`, the `LoadValues` method reads the persisted metadata and logs from the disk. This restores the node's `currentTerm`, `votedFor`, and

replicates the log into memory. If a snapshot exists, it is also loaded to restore the application state up to the last snapshot index, ensuring the node resumes operation consistent with its previous state.

4.4 Log size optimization – Log Compaction

As the system operates, the log grows indefinitely. To manage memory and disk usage, the system implements log compaction via snapshots, handled by the `Snapshot` struct in `snapshot.go`.

4.4.1 Snapshot creation mechanism (snapshot.go)

The `decideRunSnapshot` method checks if the current log size exceeds the configured `snapshotThreshold`. If the threshold is met, the following steps occur:

1. The state machine (Application) generates a serialized snapshot of its current data (e.g., the key-value map).
2. The snapshot is written to a `.snap` file using `WriteSnapshotData`.
3. The log entries covered by the snapshot (up to `lastIncludedIndex`) are truncated from the in-memory log to free resources.
4. The `lastIndex` and `lastTerm` are updated in the `Snapshot` struct to maintain log continuity.

4.4.2 Snapshot transfer protocol (InstallSnapshot RPC)

If a follower lags so far behind that the leader has already discarded the log entries needed for replication, the leader switches from sending `LogRequest` to sending `InstallSnapshot`. This is implemented in the `sendInstallSnapshotRPC` method. The snapshot is transmitted in chunks to avoid overwhelming the network. Upon receipt, the follower replaces its current state with the data provided in the snapshot and resets its log, as defined in the `InstallSnapshot` handler in `grpc.go`.

Chapter 5

System architecture

The design of the system's routing, partitioning, and consistency logic evolved through distinct iterations to balance the conflicting requirements of sub-millisecond latency (Cache) and strong data safety (Consistency). The final architecture leverages a Hybrid Control/Data Plane model, using Raft for topology management and Synchronous Replication for data propagation.

5.0.1 Initial Concept: The Gateway Model

The initial design proposed a classic "Proxy" architecture to abstract the cluster topology from the client.

Design:

- A centralized **Gateway Service** acts as the entry point for all client requests.
- The Gateway maintains the list of active storage nodes (N).
- Routing is determined by Dynamic Modular Hashing: target=hash(k)(modN).

Critique & Rejection: While simple, this model was rejected due to two critical flaws:

1. **The "Double-Hop" Latency Penalty:** Every operation requires two network traversals (Client → Gateway → Node). For an in-memory cache, this overhead is prohibitive.
2. **The "Thundering Herd" Problem:** The formula is tightly coupled to node count (N). If N changes, roughly 75% of keys are remapped, rendering the cache unusable during scaling.

5.0.2 Structural Optimization: Fixed Partitioning

To solve the "Thundering Herd" problem, the system adopts the Fixed Partition model. Keys are mapped to logical buckets rather than physical nodes. PartitionID=CRC16(key)(mod1024)

The constant 1024 ensures that adding a node results in minimal, deterministic data movement.

5.0.3 The Consistency Model: Defining "Strong Cache Consistency"

A core thesis requirement is "Strong Consistency." To achieve this without becoming a slow, disk-bound database, the architecture makes a precise distinction between Consistency and Durability.

- **Durability (Database Domain):** Guaranteeing data survives a total power failure (requires disk fsync). This is *out of scope* for this in-memory cache.
- **Consistency (Thesis Scope):** Guaranteeing that once a write is acknowledged, it is visible to all subsequent reads and survives individual node failures.

To achieve this, the system rejects "Async Replication" (Redis-style, where data loss is possible) and "Full Consensus" (CockroachDB-style, where latency is too high). Instead, it implements Synchronous Primary-Backup Replication.

5.0.4 Final Architecture: The Supervisor-Worker Model

The system is bifurcated into two distinct layers:

1. The Control Plane ("The Supervisor")

This layer consists of a small cluster (typically 3 nodes) running the Raft Consensus Algorithm.

- **Responsibility:** It manages the **Partition Table** (The Map).
- **State:** It stores the mapping: Partition P→Primary: Worker A, Backup: Worker B.
- **Role:** It acts as the "Judge." It does not store cache data. It only prevents Split-Brain scenarios by enforcing that only one valid Primary exists for any partition at any time.

2. The Data Plane ("The Workers")

This layer consists of N worker nodes that store the actual Key-Value pairs in memory. Crucially, **Worker Nodes do not run Raft**.

- **Responsibility:** High-throughput memory storage and replication.
- **Replication Factor:** The system uses N=2 (1 Primary + 1 Backup). This creates a trade-off where the system is resilient to single-node failures while minimizing network latency (compared to N=3 or higher).

3. The Write Flow (Synchronous Replication)

To guarantee Strong Consistency, the Client communicates directly with Workers using a "Smart Client" approach, enforcing a strict replication chain.

The Execution Chain:

1. **Lookup:** The Client checks its cached Partition Table to find the Primary Worker for the key.
2. **Direct Send:** The Client sends `SET key=val` to the **Primary**.
3. **Lock & Forward:** The Primary locks the key locally and immediately sends a `REPLICATE` command to the assigned **Backup Worker**.
4. **Wait for ACK:** The Primary **pauses** execution. It does not reply to the client.
5. **Backup Confirm:** The Backup writes to memory and sends an `ACK` to the Primary.
6. **Completion:** Only upon receiving the Backup's `ACK` does the Primary return `OK` to the Client.

Failure Analysis: If the Primary crashes *before* sending the final `OK`, the client treats the request as failed. If the Primary crashes *after* sending `OK`, the data is guaranteed to be on the Backup. The Raft Supervisor then detects the failure and promotes the Backup to Primary, preserving the data and maintaining Strong Consistency.

5.1 Entry point

The entry point defines how external applications interact with the distributed cache. To ensure the system meets the requirements of low latency and high availability, I analyzed two distinct approaches: the Reverse Proxy model and the Smart Client model.

5.1.1 Rejected Approach: The Reverse Proxy

In a traditional architecture, a load balancer or reverse proxy sits between the client and the cluster.

- **Mechanism:** The client sends a request to a generic address (e.g., cache.api). The proxy looks up the internal topology and forwards the request to the correct node.
- **Flaw (The Consensus Paradox):** You noted a critical flaw: *"Do proxies need consensus?"* If proxies cache the topology state to be fast, they risk having stale routes (routing to a dead node). If they don't cache state, they become a performance bottleneck. Furthermore, the proxy adds an extra network hop (Client $\rightarrow$ Proxy $\rightarrow$ Node), effectively doubling the network latency for every operation.

For these reasons, the Proxy model was rejected.

5.1.2 Selected Approach: The Smart Client

The system implements a "Smart Client" library (Driver) that is embedded directly into the user's application.

Responsibility: The Smart Client is not merely a TCP connector; it is a fully topology-aware participant in the protocol.

1. **Topology Caching:** On startup, the client connects to *any* active Worker or Supervisor and downloads the **Partition Table**.
2. **Local Routing Decision:** For a command SET key value, the client performs the partitioning logic locally:

$$target_node = PartitionTable[CRC16(key) (mod 1024)].Primary$$

3. **Direct Connection:** The client opens a persistent TCP connection directly to the target Worker Node.

Failure Correction: If the client's Partition Table is stale (e.g., a node moved recently), the Worker will reject the request with a WRONG_NODE error. The client catches this error, queries the Supervisor for a new Partition Table, and retries the operation. This guarantees that after one network round-trip, the client's view of the cluster is eventually consistent with the Raft state.

5.2 Component 2: Data Partitioning Strategy

5.2.1 The Algorithm: Fixed Slot Partitioning

To achieve predictable scaling, the system eschews dynamic modular hashing (hash(modN)) in favor of a Pre-Sharded model.

The key-space is divided into a static number of logical units called Partitions (or Slots).

- **Total Partitions (P):** Fixed at 1024.
- **Mapping Function:** PartitionID=CRC16(key)(mod1024).

5.2.2 Assignment Logic

Physical nodes do not own keys directly; they own Partitions.

- **Worker A** might own Partitions [0,340].
- **Worker B** might own Partitions [341,680].

When a new node (Worker C) joins the cluster, the system does not re-hash keys. Instead, the Supervisor simply reassigns ownership of specific partitions (e.g., taking [0,50] from A and [341,390] from B) to Worker C. This minimizes data movement and eliminates the "Thundering Herd" problem.

5.3 Component 3: The Control Plane (The Supervisor)

(Note: This replaces your "Node" section. "Supervisor" is more precise than "Node" for the Raft part.)

The Supervisor is a dedicated server (or set of 3 servers) responsible for the cluster's metadata. It acts as the "Brain" of the system.

5.3.1 The Raft Consensus Role

The Supervisor runs the Raft Consensus Algorithm. Its Finite State Machine (FSM) manages exactly one data structure: the Partition Table. State=Map⟨PartitionID→PrimaryIP,Backup

5.3.2 Responsibilities

1. **Membership Management:** Tracks which Worker nodes are alive via heartbeat monitoring.

2. **Topology Enforcement:** If a Worker dies, the Supervisor detects the failure, updates the Partition Table (promoting a Backup to Primary), and commits this change via Raft.

3. **Source of Truth:** It serves the current Partition Table to Clients and Workers upon request.

Crucially, the Supervisor never stores or processes cache data (GET/SET). This isolation ensures that heavy cache traffic cannot destabilize the consensus mechanism.

5.4 Component 4: The Data Plane (The Worker)

(Note: This replaces your "Server" and "Application" sections. This is the core implementation.)

The Worker is a high-throughput server responsible for storing data and handling client requests. Workers do **not** run the Raft algorithm.

5.4.1 Storage Engine

Each Worker maintains an in-memory hash map (e.g., Go Map or C++ Unordered Map) storing the Key-Value pairs for the partitions it currently owns.

5.4.2 Consistency Enforcement (Synchronous Replication)

To satisfy the thesis requirement of "Strong Consistency" without the overhead of Raft-per-key, Workers implement a Synchronous Primary-Backup protocol.

When a Worker receives a SET request, it executes the following atomic flow:

1. **Guard Check:** The Worker verifies against its local Partition Table that it is indeed the **Primary** for the requested key. If not, it returns WRONG_NODE.

2. **Lock & Local Write:** It locks the key and writes the value to local memory.

3. **Replication:** It identifies the **Backup Worker** for this partition and sends a REPLICATE command.

4. **Barrier:** The Worker **blocks** and waits. It does not send a response to the client.

5. **Acknowledgement:** Once the Backup Worker confirms the write (ACK), the Primary unlocks the key and returns OK to the client.

This architecture ensures that any write acknowledged to the client exists physically on at least two servers, providing durability against single-node failures.

Chapter 6

Implementation of Control Plane and Data Plane

6.1 Implementation of Control Plane

Control Node is the central orchestrator of the entire application. It is responsible for maintaining the state of the application and ensuring that all nodes are in sync.

6.1.1 Topology state machine

The core of the Controller is a Replicated State Machine that is responsible for managing the cluster metadata. This ensures that in a multi-node setup all instances of Controllers are in sync about which Data Node holds which shard.
The State represents:

- Epoch - a monotonically increasing number representing the version of the topology. This works similarly to Raft term.
- Nodes - a map of all registered Data Nodes with information about their addresses and statuses.
- Shards - a map of all shard IDs to their Primary and Replica data node IDs.

The Controller implements the Raft application interface. It processes commands like node registration and topology updates. Changes to the topology are not applied until they are committed by the Raft algorithm to ensure strong consistency. When a command is sent to the Controller, it updates its local config and, if the Controller is the leader, it may trigger a rebalance. To prevent the Raft log from growing infinitely, the system supports snapshots, saving the current cluster configuration to durable storage.

6.1.2 Node health monitoring

A dedicated component called the Reaper is responsible for monitoring the system for failures. Data nodes are designed to periodically send heartbeats to the Control Plane, which updates the timestamp of the last received heartbeat for each node. The Reaper runs a background goroutine that wakes up every second to check the liveliness of all data nodes. If a node has not sent a heartbeat within the configured grace period, it is marked as DEAD. In this case, the Reaper triggers a callback that signals the Controller to take action. If the failed node was a Primary, one of its replicas is promoted to the primary role. Subsequently, the rebalancing logic will attempt to find a new node to act as a replica to restore the desired redundancy level.

6.1.3 Shard rebalancing

Rebalancing is responsible to ensure that data availability and even distribution of data is preserved.

When a node failure is detected, the system performs an emergency promotion:

1. The controller identifies all shards where the failed node was the primary.
2. For each such shard, the first available replica is promoted to become the new primary. Since the original primary is dead, no data can be copied; the promoted replica relies on the data it already has.
3. The dead node is removed from the active node collection.

Note: When both primary node and replica dies, then the data in the shards is lost. This is a direct consequence of not providing a durable cache as noted previously. It is possible to increase number of replicas to two, but it will slow the cache, as we need to ensure that the primary data node sends data to every replica before it returns OK to the client. However, it's very unlikely to both nodes die at the same time if they are on different physical servers.

Rebalancing is triggered automatically when a new node registers or a failure occurs. It use a greedy strategy:

1. It identifies all shards that lack a primary node and assigns them to ACTIVE nodes using a round-robin approach.
2. It ensures that every shard has the required number of replicas, selecting new nodes to act as replicas if needed.

3. For initializing these new replicas or during planned load balancing, a synchronization protocol is used to ensure data consistency.

For initializing new replicas, the controller implements a 4-phase synchronization protocol:

1. The target node is instructed to perform a bulk copy of all data from the current primary node.
2. The shard is marked as LOCKED in the cluster topology. This temporarily stops writes to this specific shard to ensure a consistent state.
3. A final incremental catch-up is performed to transfer any data that was added during the initial bulk copy in phase 1.
4. The target data node is officially added to the replica set and the shard is set back to ACTIVE.

6.2 Data Plane

6.2.1 Concurrent Data Store

The data storage provides a simple but thread-safe in-memory key-value store. It uses a standard Go map structure wrapped with a mutex. The data store supports data migration. In particular it supports two functions:

1. ExportShard(shardID, totalShards) - a crucial function for rebalacing, as it iterates through the entire map and hashes each key to determine its shard and returns only those keys that belongs to given shardID
2. Import(data) - it bulks load into the map. The function is used during shard migration or recovery.

6.2.2 Lease management and fencing

Lease management is the primary system defense against split-brain problem. The data nodes are implemented to send a heartbeat to the Control Plane to inform that they are alive. If the controller acknowledges the heartbeat then the data node extends it validity timestamp. Lease is a configurable value that says how long a data node is alive. We use fencing to check the lease by comparing current time to last validity timestamp. If the difference is bigger than lease time, then the data node immediately stop serving requests. To ensure that we always check this condition, every PUT, GET, DELETE operations check the lease. If the lease has expired, then the data node will return a 503 Service Unavailable HGTTP response, effectively fencing the node out of the cluster.

6.2.3 Synchronization and Consistency

Data Plane maintains consistency by coordination with the Control Plane.

Data Plane makes sure that the topology is always synchronized. The heartbeat responses contains the latest Epoch. If the Epoch is higher than the node's local Epoch value then lease manager will immediately fetch the latest topology from the Control Plane and update the local one. This ensures that the data node is always aware of its current role that the Control Plane assigned it to - whether it's a primary node or a replica.

Data Plane uses epoch based fencing. If a node has an active lease, every write request includes an Epoch. The data node compares the client's Epoch to its own. If the requests Epoch is smaller than the local one, then the request is rejected with 412 Precondition Failed, preventing stale writes from delayed clients that have to refetch the current topology.

The last synchronization mechanism is the Primary-Backup replication. If the data node is a primary node, then its responsibility is to replicate writes to all Replica nodes. This replication is **synchronous**. This causes the system to be Strong Consistent, because we only accept the request if all replicas has saved it and the primary nodes had saved it aswell. This also provides a form of durability, as the system will keep the data as long as at least one replica is alive.

Note: The strong consistency definition is chapter 2 says that:

- If operation A completes (client gets OK), then every operation B that starts after A finishes must see the effect of A.
- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," as long as the system is consistent about that choice from then on.

So if a all replicas received the write request and then the primary node died, we have uncertainity - the second case. But since all replicas said OK - then the client will see the request as accepted from now on. In the other case if some replicas received the write and the other did not and the primary node crashed, then we have a problem of some replicas having the data and others not. This is where the client solves this problem by client-side idempotency. This will be talked in more detail in client section.

6.2.4 Data transfer protocol

Shard synchronization is a crucial part of the system. There are two data nodes that need to synchronize their data. Let's call them source and target node. The source node is the one that exports its data, while the target node is the one that imports it. Data Plane uses a pull model where:

1. The controller sends a pull request to target node
2. The target node sends a GET request to source node export endpoint that returns the result of ExportShard() function.
3. The target node calls Import() function with the data it received from source node.

6.3 Client and access library

While the Control Node handles the cluster high-level orchestration and Data Node handles lower-level data logic, the Client and Access Library is reponsible for efficient operations and ensuring reliable communication.

6.3.1 Smart client mechanism

The smart client is a implementation that eliminates the need for centrally implemented load balancer or proxy of the system that can increase latency or break the data consistency - talked in more details in chapter 3.

The client maintains a local copy of the cluster topology that is fetched from the Control Plane. The client will query the one of the controllers at the start of the session to get the latest topology. The client has to hold its own Epoch to send in every request and also compare whether the state of topology is the latest. The client also holds the list of active data nodes and shard to node mapping to connect hash to the node.

Every GET, PUT, DELETE operation has a key associated with it. The client needs to calculate the hash of the key to send the request to correct node. The client will perform operations:

1. Hash the key using FNV-1a algorithm.
2. Determine shard id by taking the hash modulo number of shards.
3. Look up the primary data node id for that shard in local cache

4. Send the request to the primary data node id.

The client is lazy but reactive. It keeps using its local cache as long as its valid. The validity of a cache is determined by a datanode response:

1. Status 412 Precondition Failed - the Epoch of the client is too old, the data node has newer Epoch in its local values

2. Status 400 Not Primary - the shard has migrated or failover occured

In both cases the client has to refetch the topology and all of the other data from Control Plane and update its local cache.

6.3.2 Client-side idempotency

To prevent duplicate operations during retries (especially when a primary node has died) the client uses idempotency token and client id fields in each request. This makes every single write and delete operation to use a unique UUID token that is used in every request. It is used to check the state of the operation. Each log in Raft algorithm is implemented to hold the idempotency token and client id. Because of this token if the client is unsure whether the operation suceeded, he can always resend the request - if the operations suceeded, nothing will happen, as the system will just find the previous request with the same token. If the operation failed, the system will retry it.

This ensures that in the Data Plane, the primary node acts as the coordinator for the token. When the writes are replicated to replicas, then the token is also passed with them. If the primary node fails after replicating but ACK-ing the client, the promoted backup already have the request with token. Then if the client is unsure whether his request has failed or not, then the replica that has become a primary node will have the request with the same token and reply OK. This solves previously talked problem where only some of the replicas receive the write and others do not because primary data node crashes. Lets imagine a situation where we have a cluster with primary node P and two replicas R_1 and R_2. The client sends a request PUT(key, value) with some token T. The primary node P replicates to R_1, which saves the request. Then P crashes. The client request times out and the client is not in the state of uncertainity, so it will resend the request. We have two possibilities:

1. R_1 becomes the new Primary node - it will see that it has the request with the token and say OK. Because of how the Shard Synchronization works, then R_2 will also have this request in it's memory, so the state is correct.

2. R_2 becomes the new Primary node - this Replica didn't receive the request, so it will not have it in it's memory. So on the resend request it will write the request to it's memory and replicate it to R_1 which will overwrite the currently held request. The state is correct

It can be easily increased to any number of Replicas, as the argument is the same.

Chapter 7

Corretness verification and Fault injection testing

7.1 Methodoly for testing distributed systems

As testing distributed systems requires independent servers I considered two options. First option was to run tests using multiple computers. This option however wasn't developer friendly. Every time I would want to test my system I had to boot multiple computers, synchronize data through version control system, then set it up. This would take too much time and effort just to run some basic testing. Thats where the second option came in - I simulated distributed servers using Docker Containers. As I can control almost everything about Docker Containers - restarts, failures, networks, this was my way of testing.

7.2 Unit and integration tests for the Raft implementation

As noted in chapter 3, Raft is a central thing in Control Plane, and Control Plane is base for everything in the system. Because of that my Raft algorithm implementation had to be throughoutly tested. I implemented unit tests for every major method in the system with edge cases. In particular I have unit tests of:

- elections - basic check whether it works on 3 and 5 nodes, then tests around replying to various Term value, then Election timeout test
- messages - creation, conversion between messages struct and grpc definitions
- persistence - saving logs, loading logs, comparing whether loaded logs are correct, saving empty logs, saving to invalid paths, saving to corrupted file

- snapshot - whether indexes in log are correct after snapshot, whether snapshot is saved correctly - after load to we have correct data, checking whether logs is truncated after snapshot, test for snapshot installation rpc, checking whether snapshot will run if below threshold or log empty, checking whether snapshot creates with any directory problems

But as far as unit tests can go, they cannot test how parts of the system behaves in terms of each other. There come integration tests:

- Full test of election, writing to a log and replication of this log to other nodes
- Test of forwarding to a leader if broadcast is sent to non-leader node
- Test of Raft recovery - whether node will have correct values after it crashes

There was also a benchmark created for Raft only. It tried to run as many requests per second as possible. On Macbook Air M1 2020, 16GB RAM it achieved around 290k requests per second.

7.3 Simulation of failures and network partitioning

While the Raft tests are useful for testing Raft, they are not useful for testing the entire system. System requirement was to be Strongly Consistent and have Partition Tolerance. Hence I wrote two tests:

- First test checked Partition Tolerance, it ran entire system and waited for it to get to some stable state (wait around 5 seconds). It then found the leader and disconnected it from the other nodes. Then the test waited for new leader election. When the new leader was elected, the new node was added to the cluster. Then the previously disconnected node was reconnected and after 5 seconds there was a check whether the reconnected node sees the newly added node.
- Second test checked Strong Consistency. It had multiple subtests in it:
  - First configured some topology of the system to look like it was running for some time to check read-after-write on it. The test run PUT request for some random value to the key and without waiting asked for the value. The expected value of read was the write value.
  - Second was to write sequence of values to the same key (change the key value multiple times). Then there was one singular read sent. The expected value was the value of the last write.
  - Third part was similar to first, but this time there were multiple keys used.

- Fourth part was about Control Plane consistency, the test tried to register a new node and expected the registration to succeed and be visible in following read
- Fifth part was to expect all Raft nodes to be available after all of these operations from previous subtests
- Sixth part was to write a value to some key and restart data-node holding it. The expected outcome was to data-node to have this value in the memory.
- Seventh part was a rerun of second part, to make sure that data-node after restart is still functioning in the intended way.
- Eight, final part was to write a value to data-node and then immediately do a read-after-write check whether replica has this data.

All of this tests suceeded, guaranteeing that the system is Strongly Consistent and Partition Tolerant.

Chapter 8

Performance analysis and Comparison with Existing Solutions

8.1 Benchmarks

The main idea about a cache is its speed. If we did not care about speed, then using database would be better. In that case, the performance analysis is crucial.
I decided to run benchmarks to calculate how many requests per second the cache can support. Benchmark works for 10 seconds, using 100 workers. It writes as many request as it can to different key each request. After ten seconds, we divide the number of total requests by 10 to get the score of requests per second. //

For benchmarks I was using Macbook Air M1, 2020 with 16GB of RAM on MacOS Tahoe 26.1.

8.2 Throughput study (RPS) depending on the number of clients and shards

Table 8.1: Client Scaling (1 Datanode)

Workers	RPS
1	5,714
10	18,855
50	28,336
100	29,605
200	27,661

Table 8.2: Horizontal Scaling (Multiple Datanodes, No Replication)

Datanodes	Combined RPS	Scaling
1	28,460	1.0x
2	55,001	1.93x
3	72,731	2.56x

Table 8.3: Replication Impact (per-node throughput)

Configuration	RPS
1 datanode (no replication)	28,909
2 datanodes (no replication)	27,100
3 datanodes (no replication)	27,468
2 datanodes (WITH replication)	26,108
3 datanodes (WITH replication)	27,657

8.3 Analysis of Raft protocol overhead on response time (Latency)

I created a benchmark for a case where we used Raft for every single operation. In reality we do not do this, because our smart client hashes the topology and doesn't need to use Raft on every request.
However in the benchmark I made the client send every request through Raft. This resulted in around 8K requests per second. Then I implemented an artificial sleep for Broadcast and LogRequest RPC operation. It accepts how many miliseconds it should sleep for before doing the part. This can showcase how important is Raft for the application. After running the benchmark scores were:

Table 8.4: Analysis of Raft protocol overhead on response time

Artificial Delay	Consensus RPS	Impact
0ms	8002	Baseline
10ms	274	29x slower
25ms	100	80x slower
50ms	59	135x slower

This shows how big of a difference can Raft speed make for the system.

8.4 Comparative study

To verify whether the score is good, we should compare it against something. I identified two open-source system best to compare against. First one is Etcd - also a Raft based implementation of key-value store that guarantees strong consistency.

However Etcd is a durable key-value store, meaning that even if nodes do crash, the data is not lost.
The second system is Redis, as it's the most used cache is the world [9]. Redis however, is an AP (Availability + Partition Tolerance) system, meaning that it doesn't guarantee strong consistency. It is eventual consistency system, made to give the best possible performance. Redis is an 15 year old, C written, highly-optimised cache. It is very hard to write something better than Redis, so it's a good comparison in terms of speed.

8.4.1 Performance comparison with Etcd

Etcd was ran in Docker. I used official package *go.etcd.io/etcd/client/v3@v3.5.11* for client.
I ran 10x 10 second, 100 workers setup described in chapter 7.1. I saved amount of requests per second for every single of these runs, then averaged it. The score was 26661 Requests Per Second.

8.4.2 Performance comparison with Redis

Redis was also ran in Docker. The client used was official *github.com/redis/go-redis/v9*. Just like in Etcd, 10x 10 second, 100 workers benchmarks were run. The score was 38776 Requests per second.

8.4.3 Conclusions from the comparative analysis

I re-run my system 10x 10 second, 100 workers benchmarks alongside Etcd and Redis. It averaged 27715 Request per second, making it 3.95% faster than Etcd and 30% slower than Redis.
This score is expected. My system should be faster than Etcd and slower than Redis. Based on analysis of Raft protocol overhead on response time, the better and more optimised my Raft implementation would be, the faster it should work. My system could be working faster, but it would never reach speed or Redis because of it's low level implementation, lenient consistency guarantee and higher level of optimizations.

Chapter 9

Summary

9.1 Evaluation of the degree of achievement of the work's objectives

My system functional and nun-functional requirements have all been meet. For each and every one of them there was a test written.

9.1.1 Encountered implementation problems and challenges

The biggest problem of the project was the implementation.
Firstly I implemented Raft with multiple mutexes. Using multiple mutexes made it harder to reason about. That caused many deadlocks that I had to debug.
My second problem with Raft implementation was that I did not start with snapshot logic instantly. This made me rewrite almost all of the logic to adjust log indexes and terms to include snapshotted terms.
Then, even after implementation of snapshots there were not efficient. My initial strategy was to send snapshot during one request. This made the request block the thread and basically hold the mutex for longer, which made the server (leader) sending snapshot to a follower unresponsible. This created a bug, where the third server, that was not receiving snapshot saw a leader as dead node and started the election. Because of correct values and logic, the server that was receiving snapshot accepted the new leader Vote Request and made the third server a new leader. My solution to this, was to split snapshot into smaller blocks. This removed unresponsibility of leader and made cluster work correctly.

9.2 Directions for further development of the project

As noted in chapter 7, the Raft speed determines entire application speed. I've found multiple tips and indentified possible directions for better performance of said algorithm. These include:

- Batching requests. Currently logs are saved every single request. This can be sped up, by batching them - sendings two or three log entires in one requests, instead of three of them. Saving logs opens and closes a file, which is a slow I/O operation. If the system would open only 33% of the time, it would greatly speed up the entire algorithm. This is however hard to implement, as if we accept a request and return OK response, then to achieve strong consistency we need to have the data in stable storage. If the node crashes before saving it (while waiting for 2/3 log entires for batch) it will completely break strong consistency.
- Saving logs to file is a slow I/O operation that reopens file multiple times. The idea is to use some heuristic to keep the file open while some variables occur. One possible variable is that, there might be way more requests and logs to save. If we did not have to close and open the file, it would make it work almost as fast as batching, but without the risk of breaking strong consistency. This is also not so easy to implement. We need to make sure that the fsync() calls are often, we also need a very good heuristic for when to close the file. If any bug occurs to the heuristic, then entire algorithm stops working as it's basing everything on log persistance.
- Based on profiler, the encoding/decoding binary writing is taking around 40% of the entire time of application. There might be introduced custom encoding and decoding that might be more specific and save everything needed in specific bits of integer, hence making the entire encoding shorter.
- The I/O operations are long, the system might benefit from using the fastest threads to handle these operations.

Chapter 10

Saved

10.1 Data partitioning

The design of any distributed system at it's conception solve the problem of partinioning the data. As the application can grow infinitely, physical limits are set. Our computer have limited capacity for memory, CPU and network I/O, problem known by vertical scaling. The solution is horizontal scaling, which is **distributing** the work on many machines and "sharding" the data.

This however introduces new challanges that are complex. We must answer some questions in design of our application:

- How will the data be divided? We must have a function f, that will map a key k to a specific node n from the set of N nodes in the cluster.
- How does client determine on which node is his data?
- What does the system do when a new node is added? What does it do when the node is removed?

10.1.1 Karger's Consistent Hashing

Proposed by Karger et al. at MIT in 1997, consistent hashing solves the "reshuffling" problem inherent in modular hashing. In a standard modulo setup ($\text{hash}(k) \pmod n$), changing n (adding/removing a node) results in nearly all keys being remapped to different nodes. Karger's approach maps both the **Nodes** and the **Keys** onto the same identifier space, typically treated as a circle or "ring" (e.g., integers 0 to $2^{32} - 1$).

- **Placement:** A node is assigned a position on the ring by hashing its ID (e.g., IP address). A key is assigned a position by hashing the key itself.

- **Lookup:** To find the node responsible for a key, we move clockwise around the ring from the key's position until we encounter a node. That node is the "coordinator" or owner of the key.

- **Scaling:** When a new node is added to the ring, it only "steals" keys from the node immediately following it in the clockwise direction. The rest of the keys in the cluster remain unaffected.

Mathematically, this ensures that when the n-th node is added, only $1/n$ of the keys need to be moved, which is the theoretical minimum amount of data movement required to maintain a balanced load.

Virtual Nodes: A pure implementation of Karger's ring often results in uneven data distribution due to the random placement of nodes. To solve this, modern implementations (like Cassandra and Dynamo) use "Virtual Nodes" (vnodes). A single physical machine acts as multiple nodes on the ring (e.g., 256 positions), ensuring a more uniform statistical distribution of keys.

10.1.2 Lamping And Veach's Jump Consistent Hash

In 2014, Google engineers Lamping and Veach introduced "Jump Consistent Hash," an algorithm that achieves the same stability properties as Karger's ring but with significantly higher performance and zero memory overhead.

Unlike Karger's approach, which requires storing the ring structure in memory (typically a binary search tree or sorted map) to look up a node, Jump Hash computes the bucket assignment algorithmically using a Pseudo-Random Number Generator (PRNG).

The logic is probability-based:

- When a cluster scales from n to $n+1$ buckets, the algorithm ensures that any specific key has exactly a $\frac{1}{n+1}$ probability of moving to the new bucket $n+1$.

- The algorithm uses the key to seed the PRNG and "jumps" forward through the sequence of bucket sizes to find the final destination.

While extremely fast (executing in nanoseconds), Jump Consistent Hash has a strict constraint: it only supports scaling at the tail. You cannot arbitrarily remove "Node 2" from a 5-node cluster; you can only scale down from 5 to 4. This makes it ideal for data storage systems where buckets are logical shards, but less flexible for direct mapping to dynamic physical servers.

10.1.3 The sorted monolithic map

This approach, also known as **Range Partitioning**, rejects the randomization of hashing entirely. Instead, it treats the entire key-space as a single, sorted monolithic map.

The data is ordered lexicographically (by byte comparison) and divided into contiguous ranges (often called “Tablets,” “Regions,” or “Ranges”).

- **Routing:** The system maintains a global index (like a B-Tree) that maps ranges to nodes. For example, keys $[A, K)$ might map to Node 1, while keys $[K, Z)$ map to Node 2.
- **Range Scans:** Unlike hash partitioning, this model allows for efficient range queries (e.g., `SCAN user:100:log:*`). Since the keys are physically stored together, the system can fetch them in a single disk seek or network call.

However, this design suffers from the **“Sequential Write” problem**. If an application generates keys with a monotonically increasing prefix (e.g., timestamps or auto-incrementing IDs), all write traffic will target the single node responsible for the end of the range, creating a massive hot spot while other nodes sit idle. This necessitates complex splitting and rebalancing logic, as seen in systems like TiKV and CockroachDB.

10.2 Analysis of standing systems

10.2.1 Redis cluster: Pre-sharded ”Hash slots”

The key-space is pre-sharded into a fixed, static number of 16,384 ”hash slots”. This number is constant regardless of the number of nodes in the cluster. A key is mapped to a slot using the formula:

$$\text{hash\_slot} = \text{CRC16(key)} \pmod{16384}$$

. Each primary node in the cluster is responsible for a subset of these 16,384 slots. A 3-node cluster, for example, might be configured as:

- Node A: Hash slots 0 - 5500
- Node B: Hash slots 5501 - 11000
- Node C: Hash slots 11001 - 16383

Load balancing is an explicit, administrator-driven process called ”resharding”. An administrator uses the redis-cli tool to specify how many slots to move from a source node to a target node. This operation can be performed with ”no downtime”.

Request Routing (Client-Side Intelligence): This is the most complex component of the Redis Cluster design. Cluster nodes do not proxy requests. If a client sends a request to a node that does not own the key's hash slot, the node replies with a redirection error. There are two types of redirection:

- MOVED: This is a permanent redirection. It informs the client that the hash slot is now permanently owned by a different node. The client must update its local cluster map and retry the request at the new node.
- ASK: This is a temporary redirection used during a resharding operation. When a slot is being migrated, it is in a MIGRATING state on the source and IMPORTING state on the target. A -ASK redirect tells the client, "The key might be on the new node, but only for this one request. Do not update your map.". The client must then send an ASKING command to the target node before sending the redirected query. This ASKING command authenticates the client to access a key in an IMPORTING slot.

Key Colocation: Redis provides a "hash tags" feature to manually force keys into the same slot. If a key contains a ... substring, only the content inside the brackets is hashed. For example, user:123:profile and user:123:account are guaranteed to be in the same hash slot, enabling multi-key operations on them.

Redis's "hash slot" model represents a pragmatic compromise. It sacrifices the automatic, self-healing nature of consistent hashing for simplicity, predictability, and explicit control. The cluster state is simply a 16,384-element array mapping slots to nodes. This is far simpler to manage, gossip, and reason about than the mathematics of a continuous ring. The trade-off is twofold: load balancing is a manual operation, and the server logic is simplified at the cost of massively complicating the client, which must implement the sophisticated -MOVED/-ASK redirection protocol.

10.2.2 Amazon Dynamo

Amazon's Dynamo, as documented in the 2007 SOSP paper, is the archetypal highly available, "always-on" key-value store.

Model: It uses the classic Karger-style consistent hashing, enhanced with virtual nodes to address load balancing and heterogeneity.

Replication & Partitioning: Dynamo's "preference list" is a key concept. For a given key k with a replication factor of N, its preference list is composed of the first N distinct physical nodes encountered by walking the ring clockwise from k's position. The first node on this list acts as the "coordinator" for writes.[1]

Load Balancing: Balancing is inherent to the virtual node model. When a

[1] `https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf`

new node is added, it assumes responsibility for numerous small virtual ranges from all other nodes on the ring, thus automatically distributing the load.

Request Routing: "Any storage node" in Dynamo is eligible to receive a client request. That node then acts as the coordinator, enforcing a quorum-like technique (e.g., $R + W > N$) among the nodes in the key's preference list.

While Dynamo's partitioning model is foundational, its overall architecture is philosophically opposed to a Raft-based system. Dynamo is the canonical "AP" (Available, Partition-Tolerant) system, designed to "sacrifice consistency under certain failure scenarios" to achieve its "always-on" goal. Its entire design—built on eventual consistency, object versioning with vector clocks, and application-assisted conflict resolution—is intended to manage and resolve data conflicts.

Raft, by contrast, is a consensus algorithm designed to build "CP" (Consistent, Partition-Tolerant) systems. Its entire purpose is to prevent conflicts and enforce strong, linearizable consistency. Therefore, while Dynamo's partitioning model is relevant, its overall goals and consistency mechanisms are a poor fit for a Raft-based project.

10.2.3 Hazelcast IMDG

Hazelcast is an in-memory data grid (IMDG) that, like Redis, uses a hash-partitioned model.

Model: The key-space is divided into a fixed, configurable number of partitions. The default is 271.

Hashing Algorithm: Hazelcast uses MurmurHash3, a high-quality, non-cryptographic hash function. The mapping is partition_id = MurmurHash3(key_bytes) mod 271. The default of 271 is chosen because it is a prime number, which helps to further ensure an even distribution of hash results.

Load Balancing: This is the most subtle and insightful aspect of Hazelcast's design. The documentation states: "Thanks to the consistent hashing algorithm, only the minimum amount of partitions are moved" when a member joins or leaves. This appears contradictory; a modulo-based system is precisely what consistent hashing was designed to replace.

This contradiction is resolved by a "Two-Level Hashing" hybrid architecture. Hazelcast uses two different hashing mechanisms for two different purposes:

- **Level 1 (Key → Partition):** This mapping is static, using MurmurHash3(key) mod 271. A given key always maps to the same partition ID, regardless of cluster size.
- **Level 2 (Partition → Member):** This mapping is dynamic. A consistent

hashing algorithm (or a similar structure) is used to map the 271 partitions (e.g., p-0 through p-270) onto the N available physical cluster members.

This two-level design provides the best of both worlds. When a new member joins, the Level 1 mapping is unchanged. The Level 2 partition-to-member consistent hash map is updated, determining that a minimal set of partitions (not keys) should be migrated to the new member. This architecture achieves the simplicity and fine-grained "virtual" units of Redis's hash slots, but adds the automatic, self-healing load balancing of Dynamo's consistent hashing.

Raft Integration (CP Subsystem): This provides a direct, practical blueprint for a Raft-based cache. By default, Hazelcast IMDG is an "AP" system, using "lazy replication" and "best-effort consistency". However, it includes an optional "CP Subsystem" that is Raft-based.

When the CP Subsystem is disabled ("unsafe mode"), CP data structures (like IAtomicLong or FencedLock) "operate in unsafe mode" and use the standard, fast, lazy-replicated partitioning. In this mode, a write "can be lost" on a crash.

When the CP Subsystem is enabled, these specific structures are linearizable, "prefer consistency over availability," and are managed by the Raft consensus mechanism.

This "bifurcated consensus model" is a critical architectural pattern. It separates the system into: 1) a high-speed, AP, partitioned data plane for the cache data itself, and 2) a separate, high-consistency, CP (Raft) control plane for coordination and metadata (e.g., locks, leader election, semaphores). This strongly suggests that using Raft for every SET operation in a cache is likely the wrong design, but using Raft to manage the cluster's state is an excellent one.

10.2.4 TiKV

TiKV is a distributed, transactional key-value store that serves as the storage layer for the TiDB database.

Model: Data is partitioned by key range.

Partitioning Unit: The basic unit of data movement and consensus is called a "Region". A Region represents a contiguous range of keys in the sorted map.

Consensus Integration: This is the core of the design. Each Region is an independent Raft group. Each Region has multiple replicas (called "Peers"), typically 3, one of which is the Raft leader that services all reads and writes for that key-range.

Load Balancing: Balancing is automatic, dynamic, and centralized, managed by the "Placement Driver (PD)".

- The PD is the "brain" of the cluster, storing the metadata for all Regions.
- It balances storage size by instructing Regions to split when they grow too large (e.g., >96–144MB) and to merge with adjacent Regions when they become too small.
- It balances load by automatically migrating Regions (Raft groups) between different nodes.
- It balances hot spots by actively transferring the Raft leadership of a hot Region to a peer on a less-loaded node.

Request Routing: A client asks the PD to identify which Region (and its leader) is responsible for a given key.

10.2.5 CockroachDB

CockroachDB is a distributed SQL database that employs an architecture very similar to TiKV's.

Model: It uses a "monolithic sorted map" of key-value pairs.

Partitioning Unit: The partitioning unit is called a "Range" (conceptually identical to TiKV's Region).

Consensus Integration: Identical to TiKV. The "replication layer" ensures that each Range is a Raft group.

Load Balancing: Automatic and size-driven. Ranges automatically split when they grow too large and merge when they become too small. The cluster automatically rebalances these ranges across all available nodes, for example, when a new node joins or an old one fails.

Request Routing (Decentralized Index): This is CockroachDB's primary architectural divergence from TiKV. It does not have a centralized PD for routing. Instead, it embeds the routing information within the key-space itself in a special index called "meta ranges".

- This index is a two-level tree: the root (meta1) points to second-level ranges (meta2), and the meta2 ranges point to the "user data" ranges.
- To find a key, a node performs a "bottom-up" lookup in this tree (which is itself composed of standard, Raft-replicated ranges), caching the results for future use.

This design represents a clear trade-off in metadata management. TiKV uses an external, centralized (though highly-available) component, the PD, which simplifies

routing logic. CockroachDB uses a decentralized, self-referential model where the metadata is "just data" stored in special ranges.

10.2.6 Google Spanner

Google's Spanner is the conceptual and technical ancestor to systems like TiKV and CockroachDB. It is a globally-distributed, synchronously-replicated database.

Model: Spanner partitions data into "tablets" (similar to Bigtable) and uses "directories" as the unit of data placement and migration.

Consensus Integration: It "shards data across many sets of Paxos state machines". (Raft is a modern, more understandable successor to the Paxos consensus algorithm; the architectural model is the same).

Load Balancing: The system is automated and managed by a hierarchy of components: a "placement driver" handles data movement across zones, while "zonemasters" assign data to "spanservers".

Request Routing: Clients use "per-zone location proxies" to find the correct spanserver serving their data.

The fact that Google (Spanner), TiKV, and CockroachDB all independently converged on the same fundamental architecture—a sorted key-space, partitioned into ranges, with each range managed by its own consensus group (Paxos/Raft)—is one of the most powerful trends in modern distributed systems. TiKV's design, for instance, is explicitly "based on the design of Google Spanner". This "Raft-per-Shard" (or "Paxos-per-Shard") model is the state-of-the-art for building strongly-consistent, scalable databases.

10.3 Node

Node is a set of Raft servers running and reaching a consensus at the same time at all times.

10.4 Server

Server is a single machine running a Raft algorithm.

10.5 Application Database

Application Database is an actual cache that we keep - this is where all the logic around saving, getting and deleting data is being done. We can access this layer

if, and only if, the Raft node reached a consensus and majority of the servers have agreed upon it.

Chapter 11

Implementation

1. Why many threads, how synchronization works between them

Chapter 12

Testing

Chapter 13

Deployment

1. Dockery 2. Use in other languages - API, installation 3. Kubernetes?

Chapter 14

Benchmarks

Chapter 15

Summary and ?Future work?

Bibliography

[1] eric brewer on cap theorem

[2] what part of raft was not formally proven

[3] usage of raft in CockroachDB

[4] usage of raft in mongodb

[5] usage of raft in neo4j

[6] usage of raft in rabbitmq

[7] Sharding name where it comes from

[8] Redis usage statistics